Optimization of a Heterogeneous Edge AI Inference Pipeline on the TDA4VM Using the C7x DSP

Emre Doğan* and Yavuz Selim Kızıltaş†
*Computer Engineering Department of Graduate School, Istanbul Technical University, Istanbul, Türkiye
Student ID: 504251571
†Graduate School of Science and Engineering, Hacettepe University, Ankara, Türkiye
Student ID: N25223483

***Abstract*—Deploying deep learning models on edge devices requires balancing high inference accuracy with strict power, thermal, and latency constraints. While standard edge CPUs or GPUs offer deployment flexibility, they often fail to meet the power-efficiency requirements of embedded real-time vision systems. The Texas Instruments TDA4VM System-on-Chip (SoC) addresses this challenge through a heterogeneous architecture featuring ARM cores, DSPs, and a Matrix Multiply Accelerator (MMA) [1], [2]. This project proposes an optimized heterogeneous inference pipeline tailored for the TDA4VM SoC, targeting low-latency and energy-efficient object detection using the C7x DSP and TIDL-based INT8 post-training quantization [3], [4]. The main goal is to minimize end-to-end inference latency under real-time, memory, and accuracy constraints by optimizing task partitioning, memory placement, and overlapped execution across heterogeneous processing units.**

***Index Terms*—edge AI, TDA4VM, C7x DSP, heterogeneous inference, INT8 quantization, TIDL, MSMC, real-time processing**

I. Introduction

Deploying deep learning models on edge devices requires balancing inference accuracy with strict latency, power, and thermal constraints. This challenge is especially significant in embedded vision systems such as Advanced Driver Assistance Systems (ADAS), robotics, and autonomous sensing platforms, where decisions must be made locally and in real time. Although general-purpose CPUs and GPUs provide flexibility, they often do not satisfy the power-efficiency and determinism requirements of embedded deployments.

The Texas Instruments TDA4VM System-on-Chip (SoC) addresses these constraints with a heterogeneous architecture consisting of ARM Cortex-A72 cores, C7x and C66x DSPs, and a Matrix Multiply Accelerator (MMA) [1], [2]. This architecture enables high-performance inference, but achieving efficient execution requires solving several system-level problems, including cross-core workload partitioning, memory movement across DDR and on-chip memory, synchronization overhead, and reduced numerical precision caused by quantization.

This project focuses on optimizing a heterogeneous object detection inference pipeline on the TDA4VM platform. Rather than proposing a new neural network architecture, the goal is to improve the system-level execution of an existing lightweight model by exploiting the hardware structure of the SoC. The primary optimization target is minimizing end-to-end inference latency, while throughput and energy efficiency are treated as secondary evaluation metrics.

II. Related Work

Deploying deep neural networks on edge computing platforms has been extensively studied to address the trade-off between inference accuracy and hardware constraints. Wang et al. [5] provide a comprehensive survey on edge intelligence, highlighting that while standard CPUs and GPUs offer flexibility, they struggle under strict power limitations, which motivates the use of heterogeneous SoC architectures. Such systems rely on coordination between general-purpose processors and specialized accelerators to achieve efficient real-time inference.

To bridge the gap between resource-intensive models and low-power edge processors, post-training quantization (PTQ) has become a widely adopted technique. Jacob et al. [6] introduced a practical INT8 quantization framework that significantly reduces memory footprint and compute cost while preserving much of the baseline accuracy. Later studies, such as Nagel et al. [7], showed that calibration-aware quantization can further reduce the accuracy loss introduced by low-precision deployment.

Memory hierarchy optimization is another critical factor in embedded inference performance. Since external DDR accesses are expensive in both latency and energy, many works emphasize the use of on-chip buffers and asynchronous data movement to overlap communication and computation. Chen et al. [8] discuss the importance of DMA-based data movement in reducing memory stalls, while Mitton et al. [9] highlight tiling and double-buffering as practical mechanisms for heterogeneous computing systems.

This project builds upon these foundations by focusing on the deployment-level optimization of a lightweight object detection model on the TDA4VM SoC [1], [2]. The contribution lies in improving task partitioning, memory placement, and stage overlap in a heterogeneous embedded inference pipeline.

III. Problem Definition

A. System / Application Description

The target system is an embedded real-time computer vision application deployed on the Texas Instruments TDA4VM SoC,

aimed at object detection tasks relevant to ADAS, robotics, and autonomous sensing. The TDA4VM integrates multiple heterogeneous processing cores—including an ARM Cortex-A72 cluster, C7x DSP with MMA, and C66x DSPs—onto a single low-power SoC [1], [2]. The application ingests camera frames, performs neural network inference to detect and classify objects, and outputs structured results (e.g., bounding boxes and class labels) in real time.

The core challenge is that the memory and compute resources of the SoC are tightly constrained: on-chip shared memory is limited, DDR bandwidth is finite, the power envelope is restricted, and the system must sustain deterministic low-latency responses in time-sensitive scenarios.

B. Objectives

The primary objective of this project is:

- **Minimize End-to-End Latency:** Reduce the total time from frame capture to final detection output while meeting real-time constraints.

The following are treated as secondary evaluation objectives:

- **Improve Throughput:** Increase sustained Frames Per Second (FPS) by overlapping pipeline stages.
- **Improve Energy Efficiency:** Improve performance-per-watt by using INT8 quantization and the specialized compute units of the SoC effectively.
- **Preserve Model Accuracy:** Keep the accuracy degradation introduced by INT8 post-training quantization at an acceptable level relative to the FP32 baseline.
- **Improve Memory Utilization:** Reduce costly DDR accesses by making effective use of on-chip shared memory for high-reuse intermediate tensors.

C. Constraints

The following constraints govern the design space of this project:

- **Hardware Constraints:** On-chip shared memory capacity is limited, and DDR bandwidth can become a bottleneck for large tensor transfers. The C7x DSP and MMA also impose architecture-specific execution and tiling constraints [2].
- **Power and Thermal Constraints:** The target platform must remain suitable for embedded deployment under sustained operation.
- **Toolchain Constraints:** Model deployment relies on the TIDL toolkit and TI Processor SDK, which support a defined subset of neural network operators in optimized INT8 mode [3], [10].
- **Accuracy Constraints:** The quantized model should remain close to the FP32 baseline in terms of detection accuracy.
- **Real-Time Constraints:** The pipeline should satisfy deterministic timing requirements and avoid excessive synchronization or memory-related delays.

D. Decision Variables

The optimization problem considers the following decision variables:

- c_i: processing unit assigned to layer or stage i
- $s_i \in \{0, 1\}$: memory placement decision for tensor or intermediate buffer i, where $s_i = 1$ denotes placement in shared on-chip memory and $s_i = 0$ denotes placement in DDR
- S_i: scheduling/order variable describing when a stage or tile is executed in the pipeline

E. Mathematical Formulation (Optimization Track)

Let $\mathcal{L} = \{l_1, l_2, \ldots, l_N\}$ denote the set of N layers in the neural network. Each layer l_i is assigned to a processing unit $c_i \in \mathcal{C} = \{\text{ARM, C7x+MMA, C66x}\}$. Let $T_i(c_i)$ denote the execution latency of layer l_i on processing unit c_i, and let m_i denote the memory footprint of the activation tensor produced by layer l_i. Let $s_i \in \{0, 1\}$ denote the memory placement of tensor i, where $s_i = 1$ indicates shared on-chip memory and $s_i = 0$ indicates DDR placement.

The primary optimization objective is to minimize total pipeline latency:

$$\min_{\{c_i\},\{s_i\},\{S_i\}} \quad T_{\text{total}} \tag{1}$$

where

$$T_{\text{total}} = \sum_{i=1}^{N} \left(T_i(c_i) + T_{\text{comm},i}(s_i) + T_{\text{sync},i}(S_i)\right) \tag{2}$$

subject to the following constraints:

$$\sum_{i:\, s_i=1} m_i \le M_{\text{shared}} \tag{3}$$

$$\Delta\text{Acc} \le \epsilon \tag{4}$$

$$T_{\text{total}} \le T_{\text{deadline}} \tag{5}$$

$$c_i \in \mathcal{C}, \quad s_i \in \{0, 1\}, \quad \forall\, i \tag{6}$$

where M_{shared} is the available on-chip shared memory capacity, ΔAcc is the accuracy drop relative to the FP32 baseline, ϵ is the maximum acceptable degradation, and T_{deadline} is the real-time latency target.

This project treats the problem as a *single-objective constrained optimization problem*: end-to-end latency is the primary objective, while throughput and energy efficiency are analyzed as secondary system-level evaluation metrics.

IV. System Architecture and Design

A. Hardware/Software Architecture

The proposed system is designed as an end-to-end partially asynchronous data pipeline. The overall heterogeneous pipeline and memory interaction are illustrated in Fig. 1. The major components of the system are listed below:

- **Data Ingestion and Pre-processing Unit (ARM Cortex-A72):** Responsible for capturing input frames from the

sensor interface and preparing the data layout required by the neural network.

- **TIDL Execution Engine (C7x DSP + MMA):** Executes the compiled INT8 neural network layers using the optimized inference kernels provided by the TIDL framework [3], [11].
- **Memory Management Unit:** Utilizes shared on-chip memory to buffer intermediate feature maps, thereby reducing costly accesses to external DDR memory. DDR-to-shared-memory transfers are managed using DMA-capable mechanisms and overlapped with computation where possible [2], [8], [9].
- **Post-processing Unit (C66x DSP / ARM):** Processes the network outputs and converts them into actionable outputs such as bounding boxes, object labels, and confidence scores.

B. Dataflow and System Model

The core optimization strategy relies on workload distribution across heterogeneous processing units. Compute-intensive tensor operations are assigned to the C7x DSP together with the MMA accelerator, while sequential tasks such as image pre-processing and lightweight post-processing are executed on the ARM Cortex-A72 or, where beneficial, on the C66x DSP cores.

Task partitioning decisions are made *statically* at compile time, based on the computational profile of each pipeline stage obtained during baseline profiling. Compute-bound operations such as convolutions and matrix multiplications are assigned to the C7x DSP and MMA, while memory-bound or control-oriented tasks such as output decoding and format conversion are assigned to the ARM or C66x cores.

The pipeline operates in a *partially asynchronous* manner. Inter-stage communication relies on *double-buffering* through shared memory regions, so that one buffer is written by the upstream stage while the downstream stage reads from the other. Synchronization between stages is achieved through event/semaphore-based coordination provided by the TI software stack, avoiding inefficient busy-wait polling [11], [12]. Data movement between DDR and shared on-chip memory is handled using DMA-based transfers issued asynchronously so that prefetching of the next input tile overlaps with processing of the current tile [2], [8], [9].

C. Requirements Analysis (Implementation Track)

1) Functional Requirements:

- The system shall accept raw image frames and perform necessary preprocessing on the ARM Cortex-A72 core.
- The system shall execute the INT8-quantized neural network model on the C7x DSP and MMA using the TIDL inference engine [3].
- The system shall produce per-frame detection outputs including bounding box coordinates, class identifiers, and confidence scores.
- The system shall support model deployment and profiling for at least one lightweight object detection architecture.
- The system shall export profiling data such as per-stage latency and memory-transfer behavior to enable analysis and optimization.

2) Non-Functional Requirements:

- **Performance:** The system should target real-time throughput with end-to-end latency compatible with 30 FPS operation.
- **Accuracy:** The deployed INT8 model should remain close to the FP32 baseline on the chosen validation dataset.
- **Memory Footprint:** The design should respect the available on-chip shared memory budget.
- **Determinism:** The pipeline should avoid unbounded blocking and maintain predictable execution behavior.
- **Portability:** The software structure should allow model replacement with limited changes to the orchestration pipeline.

D. Timing and Execution Model

The pipeline follows a frame-triggered execution model with a nominal real-time target corresponding to approximately 30 FPS. Each frame triggers a cascade of dependent tasks across heterogeneous cores, as illustrated in Fig. 1. The end-to-end latency budget is divided across major stages such as preprocessing, inference, post-processing, and synchronization/data-transfer overhead.

Inter-stage synchronization relies on event/semaphore-based coordination, ensuring that downstream stages do not begin before upstream buffers are committed [11], [12]. DMA transfers are used to overlap memory movement with C7x computation, reducing the likelihood that DDR accesses appear on the critical path [2], [8]. Worst-case and average execution times for each stage will be measured empirically during the profiling phase, and the static task assignment may be refined if any stage consistently exceeds its budget.

E. Model of Computation

The system follows a *synchronous dataflow* (SDF) interpretation at the inter-core level, where each pipeline stage consumes and produces a fixed number of tokens (frames or tiles) per firing. Within the TIDL inference stage, the C7x DSP processes the network in a layer-sequential order, tiling large feature maps to fit within the available shared-memory budget when necessary [3]. The double-buffering scheme between the preprocessing and inference stages introduces limited pipeline latency but enables overlapped execution of consecutive frames.

F. Assumptions and Design Limitations

The following assumptions and limitations apply to this project:

- **Static Workload:** The neural network graph topology and operator types are assumed to be fixed at compile time.
- **Single-Model Inference:** The pipeline is designed to execute a single detection model at a time.

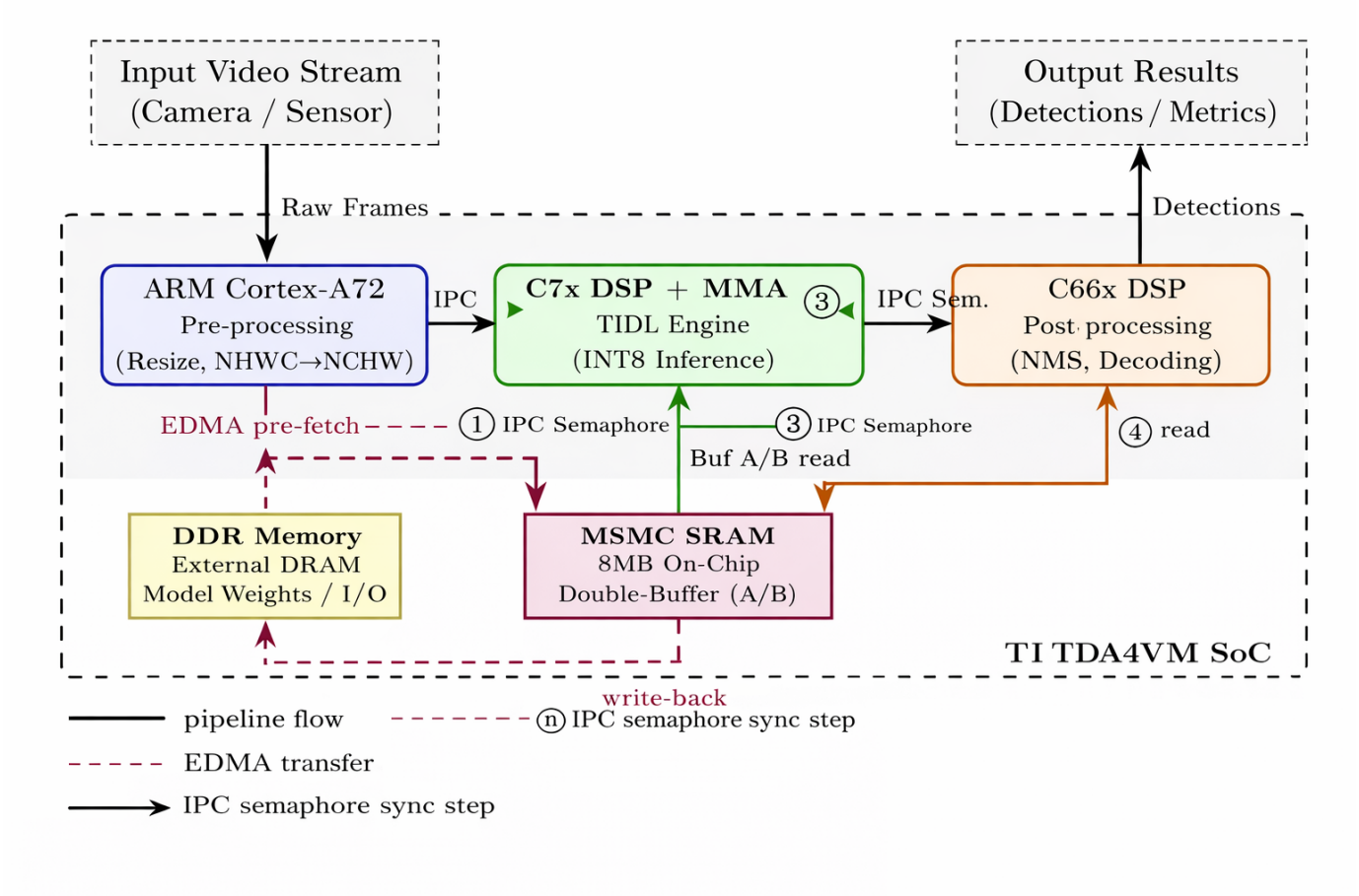

Fig. 1. Heterogeneous edge AI inference pipeline on the TI TDA4VM SoC.

- **Calibration Data Availability**: INT8 post-training quantization requires a representative calibration dataset.
- **Operator Coverage**: Not all neural network operators may be supported equally well in optimized INT8 mode by the deployment stack [3], [10].
- **Sustained Operation**: Profiling and efficiency measurements are intended to reflect sustained operation rather than only short transient behavior.

G. Resource Model

This project proposes an optimized heterogeneous inference pipeline tailored for the TDA4VM SoC. The methodology focuses on migrating a lightweight vision model to the C7x DSP. The Texas Instruments Deep Learning (TIDL) toolkit will be used to perform post-training quantization (PTQ), converting the baseline FP32 model to an INT8 format [3], [4]. Table I summarizes candidate models considered for deployment, comparing their estimated activation memory footprint against the shared-memory budget and their reported COCO mAP accuracy.

TABLE I
CANDIDATE MODEL QUICK-REFERENCE: ACTIVATION MEMORY VERSUS SHARED-MEMORY FIT AND ACCURACY.

Model	Input Size	Est. Act. Memory	Shared-Memory Fit	mAP (COCO)
YOLOX-Nano [13]	416×416	∼2–3 MB	Full	∼25
YOLOX-Tiny [13]	416×416	∼3–4 MB	Full	∼32
YOLOX-S [13]	640×640	∼6–8 MB	Marginal	∼40
SSD-MobileNetV2 [14]	300×300	∼3 MB	Full	∼22
EfficientDet-Lite0 [15]	320×320	∼3–4 MB	Full	∼26
EfficientDet-Lite1 [15]	384×384	∼5–6 MB	Likely	∼31
PeleeNet-SSD [16]	304×304	∼3–4 MB	Full	∼22

V. METHODOLOGY

A. Implementation Details (Implementation Track)

- **Milestone 1 (Weeks 1–2):** Baseline model selection, environment setup, and formal definition of optimization objectives, decision variables, and evaluation metrics.
- **Milestone 2 (Weeks 3–4):** Model export to ONNX format, preliminary benchmarking using available TI tooling, TIDL import, INT8 quantization, and baseline accuracy/performance evaluation [3], [4], [17].
- **Milestone 3 (Weeks 5–6):** Implementation of the heterogeneous inference pipeline on the target hardware, task distribution across ARM, C66x, and C7x units, and integration of shared-buffering and asynchronous data-

transfer mechanisms [2], [11], [12].

- **Milestone 4 (Weeks 7–8):** System profiling, benchmarking, bottleneck analysis, targeted memory-placement refinement, and preparation of the final report.

B. Optimization Pipeline (Optimization Track)

The optimization pipeline proceeds in four sequential phases. In the **first phase** (model preparation), a pre-trained FP32 model is selected from the candidate set in Table I and exported to the ONNX interchange format. In the **second phase** (quantization), the TIDL model import tool ingests the ONNX graph and performs INT8 post-training quantization using a representative calibration subset [3], [4]. In the **third phase** (memory placement optimization), the activation tensor memory footprint of each layer is profiled and tensors are ranked according to reuse characteristics so that high-reuse tensors can be preferentially placed in shared on-chip memory within the available capacity. In the **fourth phase** (pipeline tuning), DMA transfer sizes and double-buffer configurations are adjusted empirically to improve overlap between data transfer and C7x execution [2], [8].

C. Algorithm Description

The following pseudocode summarizes the tensor placement policy applied during the memory optimization phase. For each layer in the network, the activation tensor is characterized by its size m_i and an estimated reuse count r_i, derived from the dataflow structure of the network. Tensors are sorted in descending order of reuse count and greedily assigned to shared on-chip memory until the available capacity is exhausted; remaining tensors are mapped to DDR. This greedy policy provides a practical deployment-time approximation to the memory assignment sub-problem in constraint 3.

Input: Layer list $\mathcal{L}$, activation sizes $\{m_i\}$, reuse counts $\{r_i\}$, capacity M_{shared}
Output: Placement assignment $\{s_i\} \in \{0,1\}^N$
Sort $\mathcal{L}$ by r_i in descending order
used $\leftarrow 0$
for each layer l_i in sorted $\mathcal{L}$ **do**
  if used $+ m_i \leq M_{\text{shared}}$ **then**
    $s_i \leftarrow 1$ {Place in shared on-chip memory}
    used $\leftarrow$ used $+ m_i$
  else
    $s_i \leftarrow 0$ {Place in DDR}
  end if
end for
return $\{s_i\}$

VI. Experimental Setup

A. Platform and Tools

All experiments will be conducted on the Texas Instruments TDA4VM Evaluation Module (EVM) or an equivalent supported target platform [1]. The software stack consists of the TI Processor SDK, the TIDL runtime and model import tools, Code Composer Studio (CCS) for cross-compilation and profiling support, and relevant TI-provided runtime libraries for synchronization and data movement [10]–[12].

Prior to on-board implementation, candidate models may be screened using TI tooling that provides benchmark estimates for latency, throughput, and memory footprint [17]. This step serves as an early-stage filtering mechanism before committing to full hardware deployment.

B. Datasets / Workloads

Model accuracy is evaluated on a standard object detection benchmark such as the COCO 2017 validation set, using the standard COCO mAP metric. A representative calibration subset is used for INT8 post-training quantization. For throughput and latency benchmarking, a repeated image-stream workload is used so that reported performance reflects the inference pipeline rather than incidental host-side I/O effects.

C. Evaluation Metrics

The following metrics will be used to evaluate the proposed system:

- **End-to-End Latency (ms):** The total time from frame capture to final detection output.
- **Throughput (FPS):** Sustained frames processed per second under continuous inference load.
- **Energy Efficiency:** Performance-per-watt or platform-level power/performance indicators, if measurable.
- **Detection Accuracy (mAP):** Standard object detection mAP evaluated for both FP32 and INT8 versions of the model.
- **Memory/Transfer Efficiency:** Evidence of reduced costly DDR traffic or improved use of shared on-chip memory where measurable.

D. Comparison Cases

The evaluation will compare the following configurations:

- **Baseline 1:** Sequential execution with minimal stage overlap.
- **Baseline 2:** INT8 deployment without explicit memory-aware pipeline optimization.
- **Proposed System:** Heterogeneous pipelined execution with static task partitioning, shared-buffer overlap, and transfer/computation overlap where applicable.

VII. System-Level Considerations

Edge AI is increasingly important in autonomous and safety-relevant systems, but running accurate deep learning models on resource-constrained embedded devices remains difficult due to strict power, memory, and real-time limits. Solving this optimization problem on the TI TDA4VM platform is not only a performance problem but also a system-level engineering challenge.

Latency vs Throughput Trade-off: Pipelining can improve throughput significantly, but higher throughput does not always guarantee minimum single-frame latency. This project therefore evaluates both latency and throughput explicitly.

Memory Bandwidth and Data Movement Cost: In heterogeneous SoCs, data movement can become a dominant bottleneck. Efficient use of on-chip shared memory and overlapped transfer/computation is therefore central to the proposed design.

Synchronization Overhead: Additional pipeline stages and cross-core coordination can introduce synchronization costs that partially offset parallelism benefits. Measuring and understanding this overhead is therefore part of the project.

Accuracy vs Performance Trade-off: INT8 quantization improves efficiency but may reduce detection accuracy. The project evaluates quantization not only from a speed perspective but also in terms of accuracy preservation.

Scalability: The proposed design should remain meaningful across lightweight model variants and different input resolutions, even if absolute performance changes.

Practical Impact: In edge AI systems such as automotive vision, robotics, and autonomous sensing, improved latency and energy efficiency can contribute to faster reaction capability, lower system power consumption, and more deployable embedded solutions.

Acknowledgments

The authors thank the Hacettepe University Computer Engineering Department for providing access to relevant development resources. The TIDL toolkit and Processor SDK documentation provided by Texas Instruments were instrumental in guiding the design of the proposed pipeline.

References

[1] Texas Instruments, *TDA4VM Processors Datasheet*, Texas Instruments, 2025, rev. K, accessed 2026-04-05. [Online]. Available: https://www.ti.com/product/TDA4VM

[2] ——, *J721E DRA829/TDA4VM Processors Silicon Revision 2.0, 1.1 Technical Reference Manual*, Texas Instruments, 2025, rev. D, accessed 2026-04-05. [Online]. Available: https://www.ti.com/product/TDA4VM

[3] ——, *TI Deep Learning Product User Guide: TIDL*, Texas Instruments, 2022, online user guide for Jacinto J721E/TDA4VM, accessed 2026-04-05. [Online]. Available: https://software-dl.ti.com/jacinto7/esd/processor-sdk-rtos-jacinto7/08_05_00_11/exports/docs/tidl_j721e_08_05_00_16/ti_dl/docs/user_guide_html/index.html

[4] ——, *SDK Development Flow — Processor SDK Linux for Edge AI*, Texas Instruments, 2026, online documentation, accessed 2026-04-05. [Online]. Available: https://software-dl.ti.com/jacinto7/esd/edgeai-sdk-j721e/latest/exports/docs/development_flow.html

[5] X. Wang, Y. Han, C. Wang, Q. Zhao, X. Chen, and M. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 869–904, 2020.

[6] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[7] M. Nagel *et al.*, "Up or down? adaptive rounding for post-training quantization," in *International Conference on Machine Learning*, 2020, pp. 7197–7206.

[8] Y. Chen and W. Zhang, "An efficient dma-based data transfer engine for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 4, pp. 801–815, 2019.

[9] N. Mitton *et al.*, "On the efficiency of heterogeneous edge ai computing: A survey," *ACM Computing Surveys*, 2020.

[10] Texas Instruments, *Getting Started — Processor SDK Linux for Edge AI for J721E/TDA4VM*, Texas Instruments, 2026, online documentation, accessed 2026-04-05. [Online]. Available: https://software-dl.ti.com/jacinto7/esd/edgeai-sdk-j721e/latest/exports/docs/getting_started.html

[11] ——, *SDK Components — Edge AI Applications*, Texas Instruments, 2026, processor SDK Linux for Edge AI documentation, accessed 2026-04-05. [Online]. Available: https://software-dl.ti.com/jacinto7/esd/edgeai-sdk-j721e/latest/exports/docs/sdk_components.html

[12] ——, *Processor SDK RTOS for J721E*, Texas Instruments, 2026, online documentation, accessed 2026-04-05. [Online]. Available: https://software-dl.ti.com/jacinto7/esd/processor-sdk-rtos-jacinto7/latest/exports/docs/index.html

[13] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "Yolox: Exceeding yolo series in 2021," *arXiv preprint arXiv:2107.08430*, 2021.

[14] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 4510–4520.

[15] M. Tan, R. Pang, and Q. V. Le, "Efficientdet: Scalable and efficient object detection," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 10 781–10 790.

[16] J. Wang, Z. Li, T. Mazur, and J. Sun, "Pelee: A real-time object detection system on mobile devices," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[17] Texas Instruments, "EDGE-AI-STUDIO," Texas Instruments, 2026, official tool page, accessed 2026-04-05. [Online]. Available: https://www.ti.com/tool/EDGE-AI-STUDIO